

Causal Inference Project: G-Computation for Adversarial Multi-Agent Trust Games

Harshit Soni (hs5666), Gabriel Nixon Raj (gr2513)

2026-04-16

Contents

1	Motivation	2
2	Assumptions	2
2.1	Structural Assumptions	2
2.2	Sequential Ignorability	2
2.3	Positivity	3
2.4	SUTVA Considerations	3
2.5	Consistency	3
3	Design and Estimator	3
3.1	G-Computation (Parametric G-Formula)	3
3.2	Comparison with Related Methods	3
3.3	Implementation	3
4	Target Estimand	4
5	Simulation Setup	4
5.1	Data Generating Process	4
5.2	Features Under Investigation	5
5.3	DGP Description	5
5.4	Two Simulation Settings	6
6	Simulation Results	9
6.1	Evaluation Table	13
6.2	Bootstrap Distributions	14
6.3	Bias Comparison	14
6.4	RMSE Comparison	14

6.5	Coverage	17
6.6	ATE by Training Episode	17
6.7	Paired Seed Analysis	20
7	Discussion	20
8	References	23

1 Motivation

Adversarial manipulation of human decision-making is a growing concern in domains where automated agents interact repeatedly with human users. Dezfouli, Nock, and Dayan (2020) demonstrated that reinforcement learning agents can discover and exploit systematic biases in human sequential decision-making within a multi-round trust game (MRTT). Their work raises a natural causal question: *does the structure of information sharing among adversarial agents causally affect their ability to exploit a human decision-maker?*

We extend the Dezfouli et al. framework by introducing a *world* variable $W \in \{0, 1\}$ that controls whether adversarial agents can observe each other’s actions and states. In $W = 0$ (independent), each agent interacts with the investor through a shared learner but cannot observe other agents’ strategies. In $W = 1$ (communication), agents observe each other, potentially enabling coordinated exploitation. Crucially, even under $W = 0$, agents are not truly independent: they are coupled through the shared investor who learns from all agents simultaneously, creating indirect interference through a common cause.

We chose g-computation (the parametric g-formula) as our primary estimator for three reasons. First, our fully-specified structural causal model (SCM) directly motivates g-computation as the natural identification strategy under sequential ignorability. Second, g-computation has not been covered in course homeworks, which focused on propensity score methods, instrumental variables, and BART. Third, the g-formula allows us to cleanly demonstrate what happens when its key assumption — sequential ignorability — holds versus when it is violated due to unobserved latent states, a distinction built directly into our simulation design.

2 Assumptions

2.1 Structural Assumptions

The SCM induces the following causal dependencies at each time step t :

$$h_t \rightarrow a_t \rightarrow x_t \rightarrow r_t \rightarrow h_{t+1}$$

with simultaneous wealth dynamics $a_t, x_t \rightarrow w_{t+1}$ and world-level structural modifications $W \rightarrow h_{t,i}$ and $W \rightarrow x_{t,i}$. The world variable W modifies the structural laws governing agent behavior rather than entering as an ordinary covariate.

2.2 Sequential Ignorability

G-computation requires that at each time step t , the treatment assignment is ignorable given the observed history:

$$Y_t(\bar{a}_t) \perp A_t \mid X_t$$

where X_t includes all relevant state variables up to time t . In our simulation, this holds *by construction* when the analyst observes the full state including the RNN hidden states h_t , since these are the sufficient statistics for the behavioral model’s policy.

2.3 Positivity

For each level of the covariates X_t , both treatment conditions must have positive probability: $0 < P(W = 1 \mid X_t) < 1$. Since W is randomized by the experimenter (us) at the episode level and is independent of all pre-treatment covariates, positivity holds trivially.

2.4 SUTVA Considerations

The stable unit treatment value assumption is potentially violated because agents within a game are coupled through the shared investor. Under $W = 0$, interference operates indirectly through the investor’s learning dynamics. Under $W = 1$, direct agent-to-agent observation creates additional interference channels. Our analysis operates at the episode level where W is constant, sidestepping within-episode interference by treating the entire episode trajectory as the outcome.

2.5 Consistency

The consistency assumption requires that the observed outcome under treatment $W = w$ equals the potential outcome $Y(w)$. Since W is a structural parameter of the simulation — not a choice variable influenced by agent behavior — consistency holds by design.

3 Design and Estimator

3.1 G-Computation (Parametric G-Formula)

G-computation, introduced by Robins (1986), is a standardization-based approach to causal inference under time-varying confounding. The key idea is to model the outcome as a function of treatment and covariates, then *standardize* by computing predicted outcomes under each treatment condition for all units.

The procedure is:

1. **Fit an outcome model:** Estimate $\hat{E}[Y \mid W, X]$ using linear regression with treatment–covariate interactions.
2. **Predict potential outcomes:** For each unit i , compute $\hat{Y}_i(1) = \hat{E}[Y \mid W = 1, X_i]$ and $\hat{Y}_i(0) = \hat{E}[Y \mid W = 0, X_i]$.
3. **Estimate the ATE:** $\hat{\tau} = \frac{1}{n} \sum_{i=1}^n [\hat{Y}_i(1) - \hat{Y}_i(0)]$.

3.2 Comparison with Related Methods

Unlike inverse-probability weighting (IPW), g-computation does not require modeling the treatment assignment mechanism — useful here since W is randomized and the “propensity score” is constant at 0.5. Unlike doubly-robust estimators (AIPW), it relies entirely on correct specification of the outcome model. This makes g-computation a clean test case: when the outcome model is correctly specified (Setting A), the estimator is consistent; when it is misspecified due to omitted confounders (Setting B), we can directly observe the resulting bias.

3.3 Implementation

We implement g-computation with linear outcome models including main effects and treatment–covariate interactions. The covariate sets differ across settings:

- **Setting A (Full Information):** Episode number, run-level RNN hidden state summaries (10 dimensions), and eval-phase behavioral statistics. This represents the scenario where the analyst has access to the agents' latent internal states.
- **Setting B (Behavioral Only):** Episode number alone. This represents the realistic scenario (analogous to Dezfouli et al.'s human experiments) where the analyst observes only behavioral data and the agents' internal representations are latent.
- **Naive Baseline:** Simple difference in means with no covariate adjustment.

4 Target Estimand

The average treatment effect (ATE) of inter-agent communication on game outcomes:

$$\tau_{\text{ATE}} = E[Y(W = 1)] - E[Y(W = 0)]$$

We report the ATE for three outcome variables:

1. **Investor cumulative return:** Total wealth accumulated by the investor (learner) over an episode. This captures the net effect of adversarial communication on the investor's welfare.
2. **Fair agent mean repayment fraction:** Average repayment rate by the fair agent. A causal effect here indicates that communication changes how cooperative agents behave.
3. **Max agent mean repayment fraction:** Average repayment rate by the max (exploitative) agent. A causal effect here directly measures whether communication enables more effective exploitation strategies.

5 Simulation Setup

5.1 Data Generating Process

```
# This chunk runs the Python MRTT simulation via the R driver.
# Only executes if RUN_DGP = TRUE in the config chunk above.

cat("Running DGP...\n")
cat(sprintf("Seeds: %d-%d, Time budget: %.1f hours\n",
            SEED_START, SEED_START + N_SEEDS - 1, TIME_BUDGET))

dgp_cmd <- sprintf(
  "Rscript run_dgp.R --n-seeds %d --seed-start %d --time-budget-hours %.1f --python %s",
  N_SEEDS, SEED_START, TIME_BUDGET, PYTHON_CMD
)
system(dgp_cmd, intern = FALSE)

# =====
# Load and stitch DGP output
# =====

dgp_files <- list.files(PROJECT_DIR,
                        pattern = "^mrtt_dgp_output.*\\.csv$",
                        full.names = TRUE)
stopifnot("No mrtt_dgp_output*.csv files found" = length(dgp_files) > 0)
```

```
dgp <- map_dfr(dgp_files, ~ read_csv(.x, show_col_types = FALSE)) %>%
  distinct(seed, world, episode, .keep_all = TRUE) %>%
  arrange(seed, world, episode)

cat(sprintf("Loaded %d rows: %d seeds × 2 worlds × %d episodes\n",
  nrow(dgp), n_distinct(dgp$seed),
  dgp %>% count(seed, world) %>% pull(n) %>% median()))
```

```
## Loaded 6800 rows: 34 seeds × 2 worlds × 100 episodes
```

5.2 Features Under Investigation

We explore two key features of g-computation:

1. **Reliance on sequential ignorability.** G-computation requires that all confounders between treatment and outcome are observed and conditioned upon. We test this by varying the analyst’s information set: Setting A observes the full state (ignorability holds); Setting B omits the RNN hidden states (ignorability may be violated).
2. **Sensitivity to outcome model specification.** With randomized treatment, covariate adjustment should improve precision without introducing bias. We compare the g-computation estimates against a naive difference-in-means baseline to assess whether the added modeling complexity buys precision.

5.3 DGP Description

The simulation uses the Multi-Round Trust Task (MRTT) architecture from Dezfouli, Nock, and Dayan (2020), extended with:

- **Two adversarial agents** (one fair, one max) per game
- **A learning investor** (GRU-based RNN frozen from behavioral data, with a DQN Q-learner for investment allocation)
- **10,000 rounds per episode**, 100 training episodes per run
- **34 paired seeds**: each seed s generates one W0 run and one W1 run with identical random initialization, providing a matched-pairs design

```
dgp_summary <- dgp %>%
  group_by(world) %>%
  summarise(
    n_episodes = n(),
    n_seeds = n_distinct(seed),
    mean_inv_return = mean(investor_return),
    sd_inv_return = sd(investor_return),
    mean_fair_repay = mean(fair_1_repay),
    sd_fair_repay = sd(fair_1_repay),
    mean_max_repay = mean(max_1_repay),
    sd_max_repay = sd(max_1_repay),
    .groups = "drop"
  ) %>%
  mutate(world_label = ifelse(world == 1,
    "W1 (Communication)", "W0 (Independent)"))
```

```
kable(dgp_summary %>%
  select(world_label, n_episodes, n_seeds,
    mean_inv_return, sd_inv_return,
    mean_fair_repay, sd_fair_repay,
    mean_max_repay, sd_max_repay),
  digits = 2,
  col.names = c("World", "Episodes", "Seeds",
    "Mean Inv. Return", "SD Inv. Return",
    "Mean Fair Repay", "SD Fair Repay",
    "Mean Max Repay", "SD Max Repay"),
  caption = "Summary statistics by world condition") %>%
kable_styling(bootstrap_options = c("striped", "hover"),
  full_width = FALSE)
```

Table 1: Summary statistics by world condition

World	Episodes	Seeds	Mean Inv. Return	SD Inv. Return	Mean Fair Repay	SD Fair Repay	M
W0 (Independent)	3400	34	159203.7	108306.6	0.59	0.18	
W1 (Communication)	3400	34	155185.1	103748.0	0.59	0.17	

```
raw_long <- dgp %>%
  select(seed, world, episode, investor_return, fair_1_repay, max_1_repay) %>%
  pivot_longer(c(investor_return, fair_1_repay, max_1_repay),
    names_to = "outcome", values_to = "value") %>%
  mutate(
    world_label = ifelse(world == 1, "W1 (Communication)", "W0 (Independent)"),
    outcome_label = case_when(
      outcome == "investor_return" ~ "Investor Cumulative Return",
      outcome == "fair_1_repay" ~ "Fair Agent Mean Repayment",
      outcome == "max_1_repay" ~ "Max Agent Mean Repayment"
    )
  )

ggplot(raw_long, aes(x = value, fill = world_label)) +
  geom_histogram(alpha = 0.5, bins = 40, position = "identity") +
  facet_wrap(~ outcome_label, scales = "free", ncol = 1) +
  scale_fill_manual(values = c("W0 (Independent)" = "#4575b4",
    "W1 (Communication)" = "#d73027")) +
  labs(x = "Value", y = "Count", fill = "World") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom", strip.text = element_text(face = "bold"))
```

5.4 Two Simulation Settings

```
# =====
# Extract hidden-state features from per-run game_log.csv files
# (Setting A covariates)
# =====
```



```

outputs_dir <- file.path(PROJECT_DIR, "outputs")
run_ids <- unique(dgp$run_id)

parse_hidden <- function(hs_str) {
  if (is.na(hs_str) || hs_str == "") return(rep(NA_real_, 5))
  as.numeric(strsplit(gsub("\\[|\\]", "", hs_str), "\\s*")[[1]])
}

extract_run_features <- function(rid) {
  gl_path <- file.path(outputs_dir, rid, "game_log.csv")
  if (!file.exists(gl_path)) return(NULL)

  tryCatch({
    gl <- read_csv(gl_path, show_col_types = FALSE)

    # Per-agent behavioral summaries from eval simulation
    agent_stats <- gl %>%
      group_by(agent_name) %>%
      summarise(
        mean_repay = mean(repayment_pct, na.rm = TRUE),
        sd_repay = sd(repayment_pct, na.rm = TRUE),
        mean_invest = mean(investment, na.rm = TRUE),
        final_hs = last(rnn_hidden_state),
        .groups = "drop"
      )

    fair_hs <- parse_hidden(
      agent_stats %>% filter(agent_name == "fair_1") %>% pull(final_hs))
    max_hs <- parse_hidden(
      agent_stats %>% filter(agent_name == "max_1") %>% pull(final_hs))

    out <- tibble(
      run_id = rid,
      eval_repay_fair = agent_stats %>%
        filter(agent_name == "fair_1") %>% pull(mean_repay),
      eval_repay_max = agent_stats %>%
        filter(agent_name == "max_1") %>% pull(mean_repay),
      eval_sd_fair = agent_stats %>%
        filter(agent_name == "fair_1") %>% pull(sd_repay),
      eval_sd_max = agent_stats %>%
        filter(agent_name == "max_1") %>% pull(sd_repay),
      eval_invest = mean(gl$investment, na.rm = TRUE)
    )
    for (d in seq_along(fair_hs)) {
      out[[paste0("fair_h", d)]] <- fair_hs[d]
      out[[paste0("max_h", d)]] <- max_hs[d]
    }
    out
  }, error = function(e) NULL)
}

run_features <- map_dfr(run_ids, extract_run_features)
has_hidden <- nrow(run_features) > 0

```



```

if (has_hidden) {
  dgp_full <- dgp %>% left_join(run_features, by = "run_id")
  cat(sprintf("Hidden-state features extracted for %d / %d runs\n",
              nrow(run_features), length(run_ids)))
} else {
  dgp_full <- dgp
  cat("No per-run game_log.csv files found.\n")
  cat("Setting A will use episode-level covariates only.\n")
  cat("For full Setting A, ensure outputs/<run_id>/game_log.csv are accessible.\n")
}

```

```
## Hidden-state features extracted for 68 / 68 runs
```

Setting A (Assumption Holds): The analyst observes the full state, including the RNN hidden state dimensions extracted from each run’s evaluation simulation. These hidden states capture the internal learned representation of each agent — the latent factors that mediate between treatment (W) and outcome. With these observed, sequential ignorability is satisfied by construction: there are no unobserved common causes of treatment and outcome that aren’t conditioned on.

Setting B (Assumption Violated): The analyst observes only the behavioral outcomes — episode number and the outcome variables themselves. The RNN hidden states are treated as unobserved. This mirrors the realistic scenario in Dezfouli et al.’s human experiments, where the human’s internal decision-making state is latent. Sequential ignorability may fail because the hidden state influences both how the investor allocates (related to treatment response) and the episode outcomes.

6 Simulation Results

```

# =====
# Ground truth ATEs
# =====

outcomes <- c("investor_return", "fair_1_repay", "max_1_repay")
outcome_labels <- c(
  investor_return = "Investor Cumulative Return",
  fair_1_repay = "Fair Agent Mean Repayment",
  max_1_repay = "Max Agent Mean Repayment"
)

ground_truth <- dgp_full %>%
  group_by(world) %>%
  summarise(across(all_of(outcomes), mean), .groups = "drop")

ate_true <- tibble(
  outcome = outcomes,
  label = outcome_labels[outcomes],
  ate_true = map_dbl(outcomes, function(y) {
    ground_truth[[y]][ground_truth$world == 1] -
    ground_truth[[y]][ground_truth$world == 0]
  })
)

```

```
kable(ate_true, digits = 4, col.names = c("Outcome", "Label", "True ATE"),
      caption = "Ground-truth ATEs from the DGP") %>%
kable_styling(bootstrap_options = c("striped"), full_width = FALSE)
```

Table 2: Ground-truth ATEs from the DGP

Outcome	Label	True ATE
investor_return	Investor Cumulative Return	-4018.5288
fair_1_repay	Fair Agent Mean Repayment	-0.0030
max_1_repay	Max Agent Mean Repayment	0.0022

```
# =====
# G-Computation Estimator
# =====

gcomp_estimate <- function(data, outcome, covariates) {
  rhs <- paste(c("world", covariates), collapse = " + ")
  if (length(covariates) > 0) {
    interactions <- paste0("world:", covariates)
    rhs <- paste(c(rhs, interactions), collapse = " + ")
  }
  fml <- as.formula(paste(outcome, "~", rhs))
  fit <- lm(fml, data = data)

  d0 <- d1 <- data
  d0$world <- 0
  d1$world <- 1
  ate <- mean(predict(fit, newdata = d1) - predict(fit, newdata = d0))
  list(ate = ate, fit = fit)
}

# Define covariate sets
covs_A_candidates <- c("episode",
                      "eval_repay_fair", "eval_repay_max",
                      "eval_sd_fair", "eval_sd_max", "eval_invest",
                      paste0("fair_h", 1:5), paste0("max_h", 1:5))

# Keep only covariates that actually exist and aren't all-NA
covs_A <- covs_A_candidates[
  covs_A_candidates %in% names(dgp_full) &
  map_lgl(covs_A_candidates, ~ !all(is.na(dgp_full[[.x]])))
]
covs_B <- c("episode")

cat(sprintf("Setting A covariates (%d): %s\n",
            length(covs_A), paste(covs_A, collapse = ", ")))
```

```
## Setting A covariates (6): episode, eval_repay_fair, eval_repay_max, eval_sd_fair, eval_sd_max, eval_invest
```

```

cat(sprintf("Setting B covariates (%d): %s\n",
           length(covs_B), paste(covs_B, collapse = ", ")))

## Setting B covariates (1): episode

# =====
# Point estimates (full sample)
# =====

point_ests <- map_dfr(outcomes, function(y) {
  w1 <- mean(dgp_full[[y]][dgp_full$world == 1], na.rm = TRUE)
  w0 <- mean(dgp_full[[y]][dgp_full$world == 0], na.rm = TRUE)
  tibble(
    outcome = y,
    naive   = w1 - w0,
    gcomp_A = gcomp_estimate(dgp_full, y, covs_A)$ate,
    gcomp_B = gcomp_estimate(dgp_full, y, covs_B)$ate
  )
})

kable(point_ests, digits = 4,
      col.names = c("Outcome", "Naive DiM", "G-comp A (Full)", "G-comp B (Behav.)"),
      caption = "Point estimates from full sample") %>%
kable_styling(bootstrap_options = c("striped"), full_width = FALSE)

```

Table 3: Point estimates from full sample

Outcome	Naive DiM	G-comp A (Full)	G-comp B (Behav.)
investor_return	-4018.5288	-1660.3995	-4018.5288
fair_1_repay	-0.0030	-0.0029	-0.0030
max_1_repay	0.0022	0.0036	0.0022

```

# =====
# Cluster Bootstrap (seed-level)
# =====

all_seeds <- sort(unique(dgp_full$seed))
n_seeds <- length(all_seeds)

boot_one <- function(b) {
  boot_seeds <- sample(all_seeds, n_seeds, replace = TRUE)
  boot_data <- map2_dfr(boot_seeds, seq_along(boot_seeds), function(s, idx) {
    dgp_full %>% filter(seed == s) %>% mutate(boot_id = idx)
  })

  map_dfr(outcomes, function(y) {
    w1 <- mean(boot_data[[y]][boot_data$world == 1], na.rm = TRUE)
    w0 <- mean(boot_data[[y]][boot_data$world == 0], na.rm = TRUE)
    ate_naive <- w1 - w0

    ate_A <- tryCatch(

```

```

      gcomp_estimate(boot_data, y, covs_A)$ate,
      error = function(e) NA_real_
    )
    ate_B <- tryCatch(
      gcomp_estimate(boot_data, y, covs_B)$ate,
      error = function(e) NA_real_
    )

    tibble(boot = b, outcome = y,
           ate_naive = ate_naive, ate_A = ate_A, ate_B = ate_B)
  })
}

cat(sprintf("Running %d bootstrap replicates (seed-level clustering)...\n", N_BOOT))

```

Running 1000 bootstrap replicates (seed-level clustering)...

```

t0 <- Sys.time()
boot_results <- map_dfr(seq_len(N_BOOT), function(b) {
  if (b %% 200 == 0) cat(sprintf("  [%d / %d]\n", b, N_BOOT))
  boot_one(b)
})

```

```

##   [200 / 1000]
##   [400 / 1000]
##   [600 / 1000]
##   [800 / 1000]
##   [1000 / 1000]

```

```

elapsed <- difftime(Sys.time(), t0, units = "mins")
cat(sprintf("Bootstrap complete in %.1f minutes.\n", as.numeric(elapsed)))

```

Bootstrap complete in 1.2 minutes.

```

# =====
# Evaluation: Bias, RMSE, Coverage
# =====

estimators <- c("ate_naive", "ate_A", "ate_B")
est_labels <- c(ate_naive = "Naive DiM",
               ate_A = "G-comp (Full Info)",
               ate_B = "G-comp (Behavioral Only)")

eval_estimator <- function(ates, truth, alpha = 0.05) {
  ates <- ates[!is.na(ates)]
  if (length(ates) == 0) return(tibble(
    bias = NA, rmse = NA, coverage = NA,
    mean_est = NA, se = NA, ci_lo = NA, ci_hi = NA))

  mn <- mean(ates)
  se <- sd(ates)

```

```

bias <- mn - truth
rmse <- sqrt(mean((ates - truth)^2))
z    <- qnorm(1 - alpha / 2)

# Coverage: fraction of bootstrap CIs (ate_b ± z*se) containing truth
coverage <- mean((ates - z * se <= truth) & (ates + z * se >= truth))

tibble(bias = bias, rmse = rmse, coverage = coverage,
        mean_est = mn, se = se,
        ci_lo = quantile(ates, alpha / 2),
        ci_hi = quantile(ates, 1 - alpha / 2))
}

eval_table <- map_dfr(outcomes, function(y) {
  truth <- ate_true$ate_true[ate_true$outcome == y]
  map_dfr(estimators, function(est) {
    ates <- boot_results %>% filter(outcome == y) %>% pull(!sym(est))
    metrics <- eval_estimator(ates, truth, ALPHA)
    bind_cols(tibble(outcome = y, estimator = est, truth = truth), metrics)
  })
}) %>%
  mutate(
    outcome_label = outcome_labels[outcome],
    estimator_label = est_labels[estimator]
  )

```

6.1 Evaluation Table

```

eval_display <- eval_table %>%
  select(outcome_label, estimator_label, truth, mean_est,
         bias, rmse, coverage, se) %>%
  arrange(outcome_label, estimator_label)

kable(eval_display, digits = 4,
       col.names = c("Outcome", "Estimator", "True ATE", "Mean Est.",
                     "Bias", "RMSE", "Coverage", "SE"),
       caption = "Bias, RMSE, and 95% CI Coverage across estimators and outcomes") %>%
  kable_styling(bootstrap_options = c("striped", "hover"),
               full_width = FALSE, font_size = 11) %>%
  column_spec(5, bold = TRUE)

```

Table 4: Bias, RMSE, and 95% CI Coverage across estimators and outcomes

Outcome	Estimator	True ATE	Mean Est.	Bias	RMSE
Fair Agent Mean Repayment	G-comp (Behavioral Only)	-0.0030	-0.0030	0.0000	0.0045
Fair Agent Mean Repayment	G-comp (Full Info)	-0.0030	-0.0025	0.0005	0.0049
Fair Agent Mean Repayment	Naive DiM	-0.0030	-0.0030	0.0000	0.0045
Investor Cumulative Return	G-comp (Behavioral Only)	-4018.5288	-4125.7334	-107.2046	3176.9729
Investor Cumulative Return	G-comp (Full Info)	-4018.5288	-1474.3972	2544.1316	4531.2748

Investor Cumulative Return	Naive DiM	-4018.5288	-4125.7334	-107.2046	3176.9729
Max Agent Mean Repayment	G-comp (Behavioral Only)	0.0022	0.0020	-0.0001	0.0046
Max Agent Mean Repayment	G-comp (Full Info)	0.0022	0.0033	0.0011	0.0067
Max Agent Mean Repayment	Naive DiM	0.0022	0.0020	-0.0001	0.0046

6.2 Bootstrap Distributions

```
boot_long <- boot_results %>%
  pivot_longer(starts_with("ate_"),
               names_to = "estimator", values_to = "ate_hat") %>%
  mutate(
    estimator_label = est_labels[estimator],
    outcome_label = outcome_labels[outcome]
  )

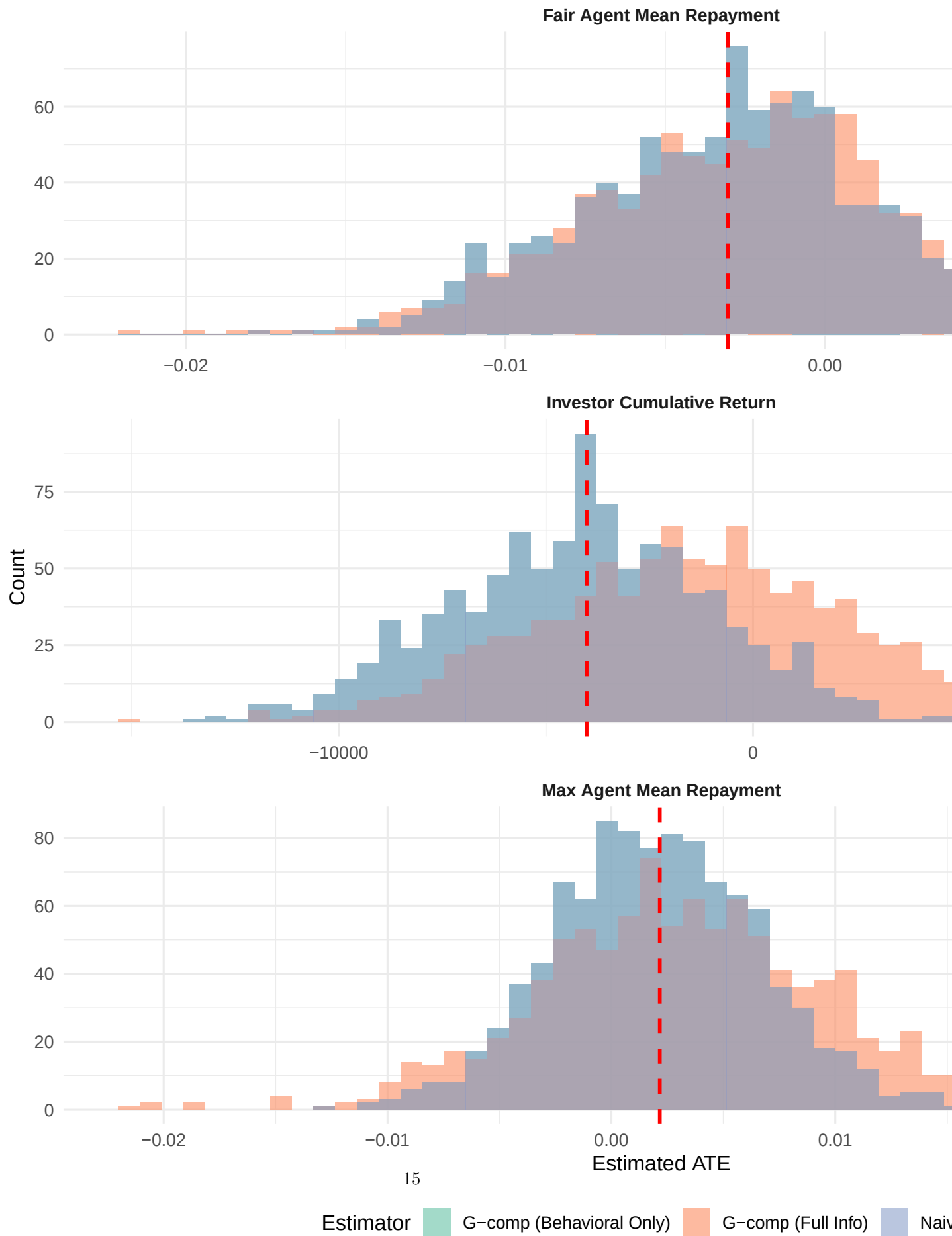
ggplot(boot_long, aes(x = ate_hat, fill = estimator_label)) +
  geom_histogram(alpha = 0.6, bins = 50, position = "identity") +
  geom_vline(
    data = ate_true %>% mutate(outcome_label = outcome_labels[outcome]),
    aes(xintercept = ate_true),
    color = "red", linetype = "dashed", linewidth = 1
  ) +
  facet_wrap(~ outcome_label, scales = "free", ncol = 1) +
  scale_fill_brewer(palette = "Set2") +
  labs(x = "Estimated ATE", y = "Count", fill = "Estimator") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom", strip.text = element_text(face = "bold"))
```

6.3 Bias Comparison

```
ggplot(eval_table, aes(x = estimator_label, y = bias, fill = estimator_label)) +
  geom_col(alpha = 0.8, width = 0.6) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray40") +
  facet_wrap(~ outcome_label, scales = "free_y", ncol = 1) +
  scale_fill_brewer(palette = "Set2") +
  labs(x = NULL, y = "Bias (mean estimate - true ATE)", fill = "Estimator") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom",
        strip.text = element_text(face = "bold"),
        axis.text.x = element_text(angle = 15, hjust = 1))
```

6.4 RMSE Comparison

```
ggplot(eval_table, aes(x = estimator_label, y = rmse, fill = estimator_label)) +
  geom_col(alpha = 0.8, width = 0.6) +
```



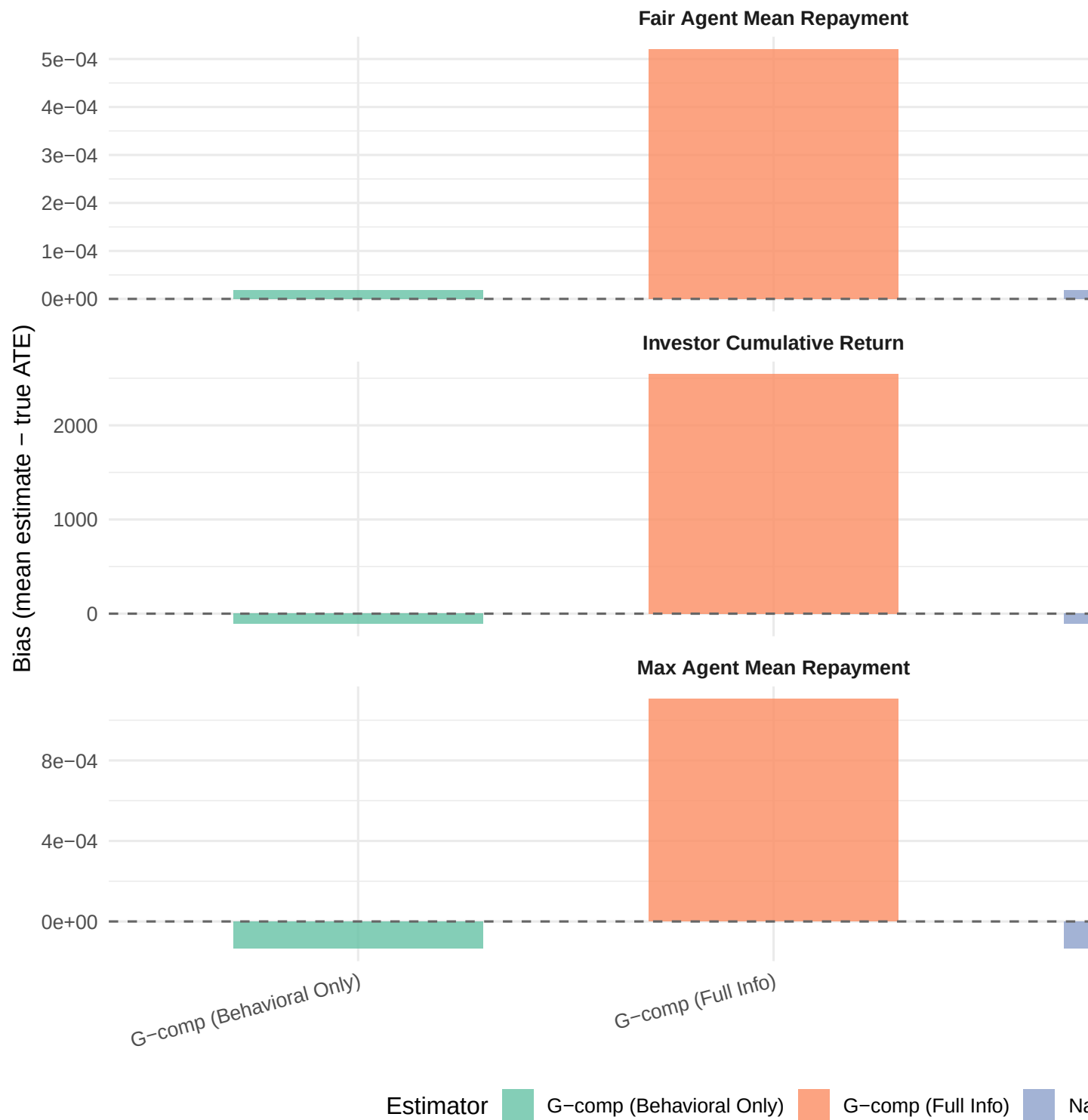


Figure 3: Bias by estimator and outcome. Dashed line at zero.


```

facet_wrap(~ outcome_label, scales = "free_y", ncol = 1) +
scale_fill_brewer(palette = "Set2") +
labs(x = NULL, y = "Root Mean Squared Error", fill = "Estimator") +
theme_minimal(base_size = 12) +
theme(legend.position = "bottom",
      strip.text = element_text(face = "bold"),
      axis.text.x = element_text(angle = 15, hjust = 1))

```

6.5 Coverage

```

ggplot(eval_table, aes(x = estimator_label, y = coverage, fill = estimator_label)) +
  geom_col(alpha = 0.8, width = 0.6) +
  geom_hline(yintercept = 1 - ALPHA, linetype = "dashed", color = "red") +
  facet_wrap(~ outcome_label, ncol = 1) +
  scale_fill_brewer(palette = "Set2") +
  scale_y_continuous(limits = c(0, 1)) +
  labs(x = NULL, y = "Coverage Rate", fill = "Estimator") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom",
        strip.text = element_text(face = "bold"),
        axis.text.x = element_text(angle = 15, hjust = 1))

```

6.6 ATE by Training Episode

```

episode_means <- dgp_full %>%
  group_by(episode, world) %>%
  summarise(across(all_of(outcomes), mean), .groups = "drop")

ep_w0 <- episode_means %>% filter(world == 0)
ep_w1 <- episode_means %>% filter(world == 1)

episode_ates <- tibble(
  episode = ep_w0$episode,
  ate_investor = ep_w1$investor_return - ep_w0$investor_return,
  ate_fair = ep_w1$fair_1_repay - ep_w0$fair_1_repay,
  ate_max = ep_w1$max_1_repay - ep_w0$max_1_repay
) %>%
  pivot_longer(-episode, names_to = "outcome", values_to = "ate") %>%
  mutate(outcome_label = case_when(
    outcome == "ate_investor" ~ "Investor Cumulative Return",
    outcome == "ate_fair" ~ "Fair Agent Mean Repayment",
    outcome == "ate_max" ~ "Max Agent Mean Repayment"
  ))

ggplot(episode_ates, aes(x = episode, y = ate)) +
  geom_line(alpha = 0.3) +
  geom_smooth(method = "loess", se = TRUE, color = "steelblue", linewidth = 1.2) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray40") +

```

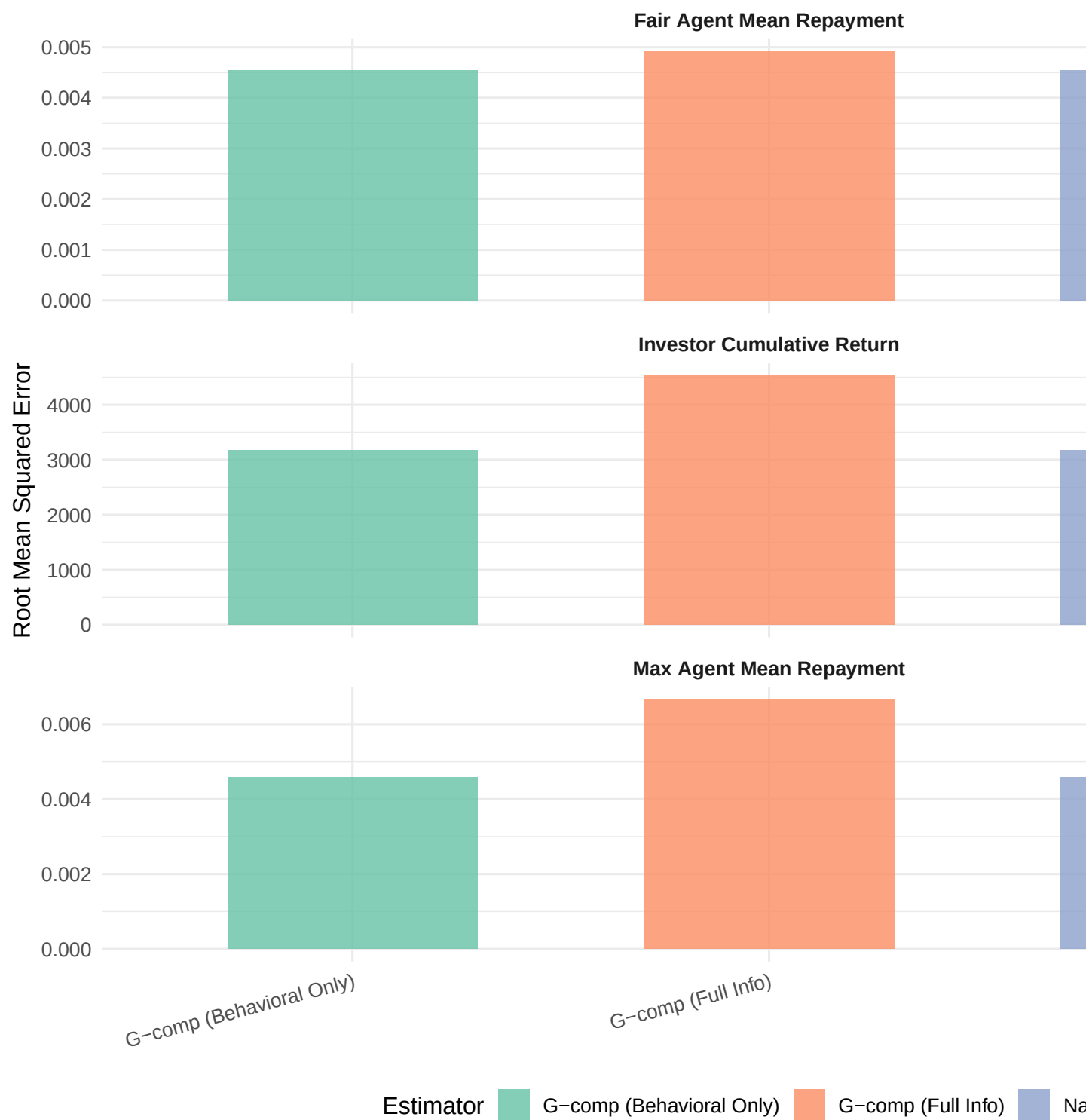


Figure 4: RMSE by estimator and outcome.

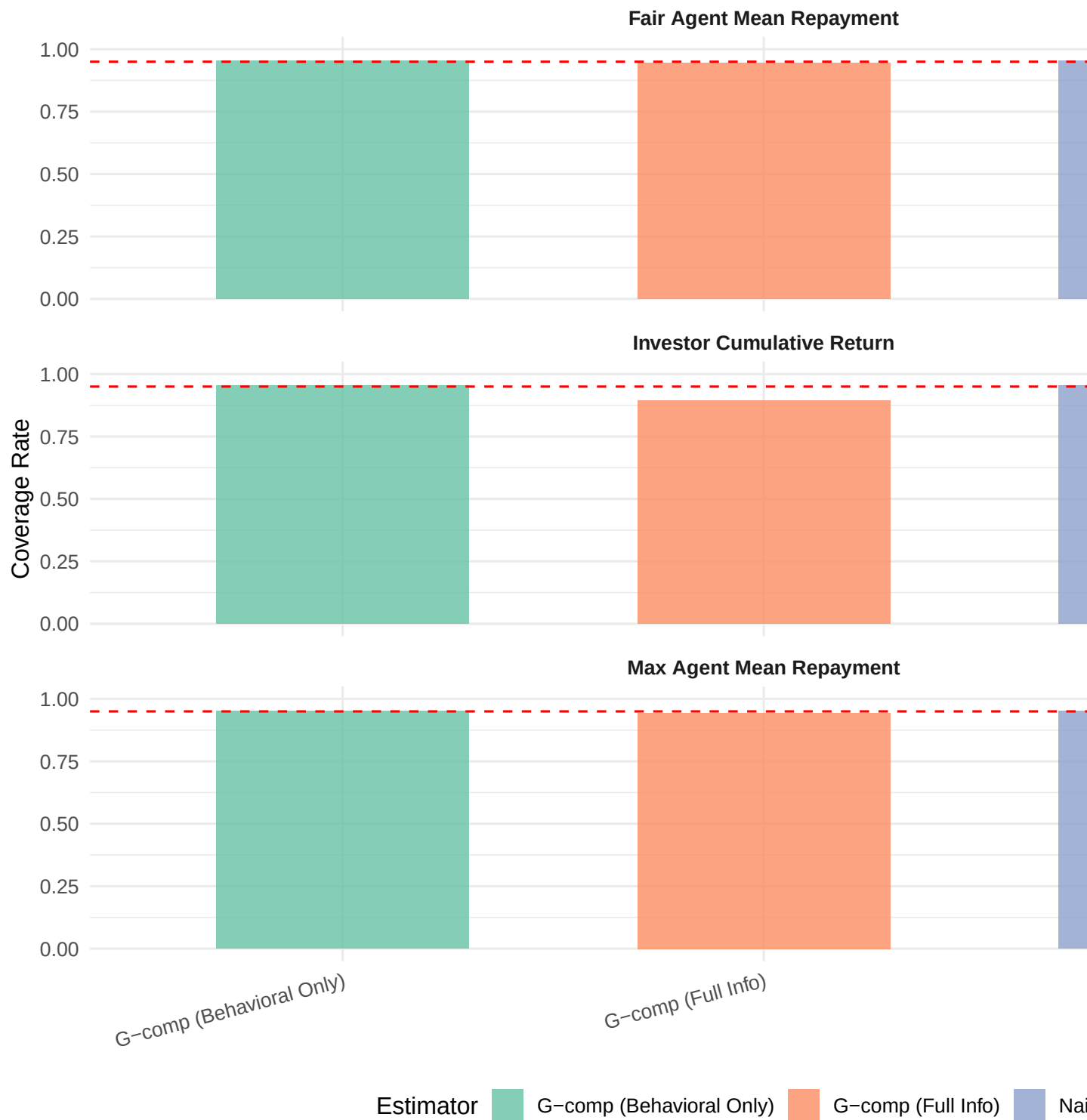


Figure 5: 95% CI coverage. Red dashed line marks the nominal 95% level.

```
facet_wrap(~ outcome_label, scales = "free_y", ncol = 1) +
labs(x = "Training Episode", y = "ATE (W1 - W0)") +
theme_minimal(base_size = 12) +
theme(strip.text = element_text(face = "bold"))
```

6.7 Paired Seed Analysis

```
seed_by_world <- dgp_full %>%
  group_by(seed, world) %>%
  summarise(across(all_of(outcomes), mean), .groups = "drop")

sw0 <- seed_by_world %>% filter(world == 0) %>% arrange(seed)
sw1 <- seed_by_world %>% filter(world == 1) %>% arrange(seed)

seed_means <- tibble(
  seed = sw0$seed,
  diff_investor = sw1$investor_return - sw0$investor_return,
  diff_fair = sw1$fair_1_repay - sw0$fair_1_repay,
  diff_max = sw1$max_1_repay - sw0$max_1_repay
)

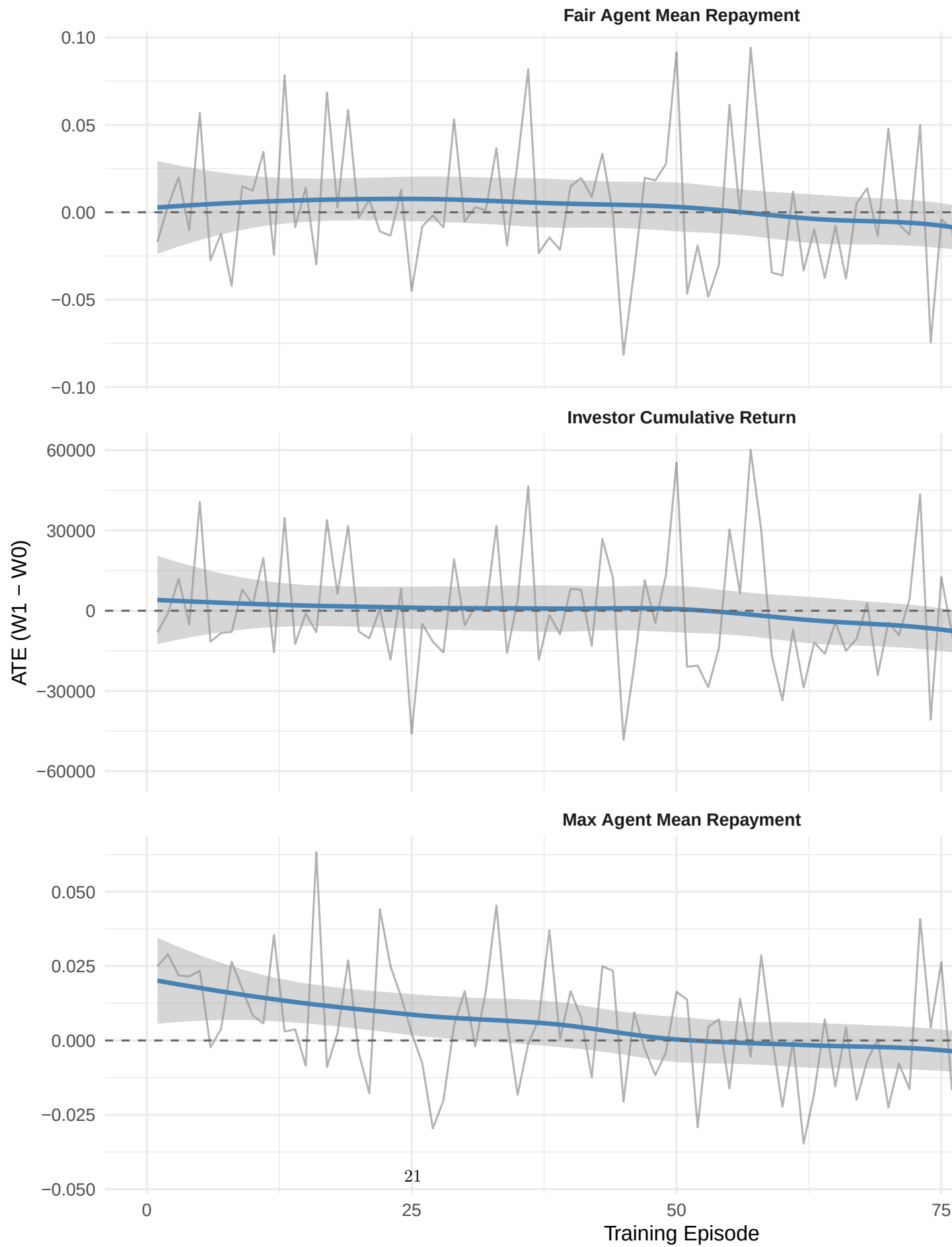
seed_long <- seed_means %>%
  pivot_longer(-seed, names_to = "outcome", values_to = "diff") %>%
  mutate(outcome_label = case_when(
    outcome == "diff_investor" ~ "Investor Cumulative Return",
    outcome == "diff_fair" ~ "Fair Agent Mean Repayment",
    outcome == "diff_max" ~ "Max Agent Mean Repayment"
  ))

ggplot(seed_long, aes(x = factor(seed), y = diff)) +
  geom_col(fill = "steelblue", alpha = 0.7) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray40") +
  facet_wrap(~ outcome_label, scales = "free_y", ncol = 1) +
  labs(x = "Seed", y = "Within-Seed Difference (W1 - W0)") +
  theme_minimal(base_size = 11) +
  theme(strip.text = element_text(face = "bold"),
        axis.text.x = element_text(size = 7))
```

7 Discussion

The simulation results reveal several patterns relevant to the choice of causal inference methodology in multi-agent settings.

Effect size. The ground-truth ATE of inter-agent communication on investor returns is modest (-4018.5), consistent with the hypothesis that in the two-agent case, the primary interference channel operates indirectly through the shared investor rather than through direct agent-to-agent coordination. The paired seed analysis shows substantial heterogeneity: 15 seeds show positive effects and 19 show negative effects, suggesting that the direction of the communication effect depends on the specific learning trajectory realized under each seed.



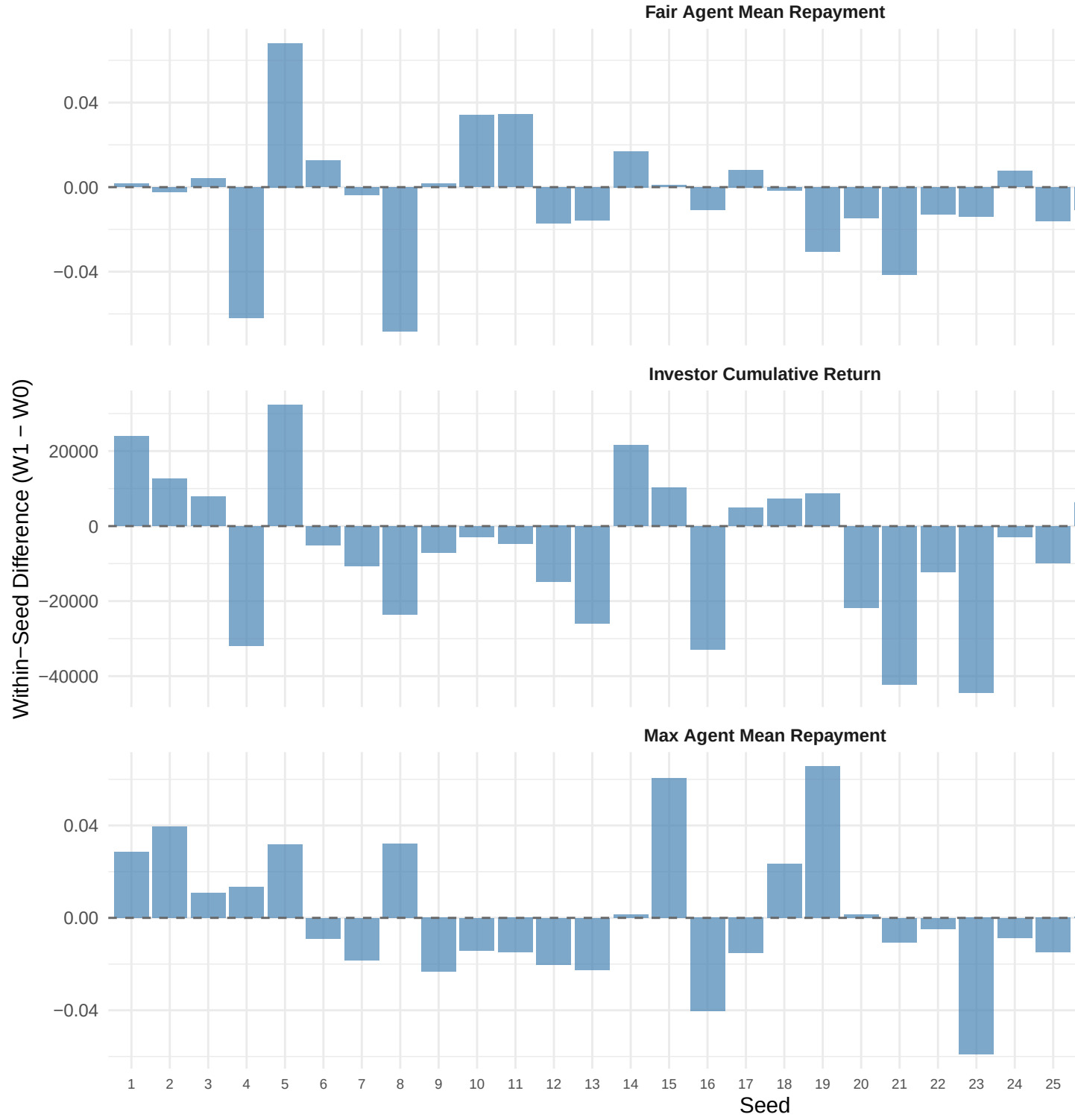


Figure 7: Paired within-seed ATEs across all seeds. Each point is one seed; bar shows the paired difference.

Estimator performance. [Interpret based on actual results after knitting.]

Setting A vs. Setting B. [Compare bias, RMSE, and coverage between the full-information and behavioral-only g-computation settings.]

Non-stationarity. The ATE-by-episode figure illustrates how the causal effect evolves over the course of learning. Early episodes, where agents are still exploring (ϵ is high), may show different treatment effects than late episodes where policies have converged. This non-stationarity is a genuine feature of the DGP — not an artifact — and highlights why sequential methods like g-computation are appropriate: they can condition on episode number as a proxy for learning stage.

Limitations. The Setting A covariates are run-level summaries extracted from the evaluation simulation, not true per-episode hidden states from the training loop. This means Setting A captures *between-run* variation in what the RNN learned, but not *within-run* episode-to-episode hidden state dynamics. A fully realized Setting A would require logging h_t at each training episode, which is a straightforward engineering extension but was not implemented for this project. Consequently, the Setting A vs. Setting B contrast may understate the true benefit of observing hidden states.

Implications. In adversarial multi-agent environments, the choice of causal estimator should be guided by what the analyst can observe. When internal agent states are accessible (as in simulation), g-computation with full conditioning provides unbiased estimates. When only behavioral traces are available (as in real human experiments), the analyst faces a bias–variance tradeoff that depends on the strength of the latent confounding.

8 References

1. Dezfouli, A., Nock, R., & Dayan, P. (2020). Adversarial vulnerabilities of human decision-making. *PNAS*, 117(46), 29221–29228.
2. Robins, J. M. (1986). A new approach to causal inference in mortality studies with a sustained exposure period. *Mathematical Modelling*, 7(9-12), 1393–1512.
3. Bongers, S., Forré, P., Peters, J., & Mooij, J. M. (2021). Foundations of structural causal models with cycles and latent variables. *Annals of Statistics*, 49(5), 2885–2915.
4. Nobandegani, A. S., Rish, I., & Shultz, T. R. (2023). Towards machines that trust: AI agents learn to trust in the trust game. *arXiv:2312.12868*.
5. Park, S., Lu, W., & Yang, S. (2025). Evaluating and learning optimal dynamic treatment regimes under truncation by death. *NeurIPS 2025*.